

Corrosion Detection GenAI

Vision + LLM Inspection Assistant

🚀 Project Overview

The **Corrosion Detection GenAI System** is a multimodal AI solution that automates the detection and analysis of corrosion in industrial assets such as pipelines, tanks, beams, and structural platforms.

It combines:

- 🖼️ **Computer Vision models (YOLO / CNN / Vision Transformers)**
- 🗣️ **Large Language Models (LLMs)**
- 💬 **Interactive AI Chat Assistant**

The system not only detects corrosion from images but also generates **structured inspection reports, severity analysis, risk levels, and maintenance recommendations** using generative AI.

🔍 Problem Statement

Manual inspection of industrial assets for corrosion is:

- Time-consuming
- Highly dependent on human expertise
- Inconsistent across inspectors
- Prone to missed defects in large-scale infrastructure

💡 Solution

This system automates corrosion detection using AI and enhances it with GenAI-based reasoning to:

- Detect corrosion in images
 - Classify severity levels
 - Generate inspection reports
 - Provide maintenance recommendations
 - Answer follow-up questions via chat interface
-

System Architecture

```
graph TD
    A[Frontend (React + Vite)] --> B[Image Upload Interface]
    B --> C[FastAPI Backend]
    C --> D["Vision Model (YOLO / CNN / ViT)"]
    D --> E["Detection Outputs (bounding boxes + severity)"]
    E --> F["LLM Layer (GPT-style reasoning)"]
    F --> G[Structured Report Generation]
    G --> H[Chat Assistant (Q&A over results)]
```

Workflow

1. User selects industrial asset (pipeline, tank, structure)
 2. Uploads image(s) for inspection
 3. Image preprocessing (resize, normalization)
 4. Vision model performs:
 - Corrosion detection
 - Severity classification
 5. Output passed to LLM
 6. LLM generates:
 - Inspection report
 - Risk level
 - Maintenance suggestions
 7. Multimodal fusion combines vision + text insights
 8. User interacts with AI chat assistant for follow-up queries
-

Key Features

- 🔍 Automated corrosion detection from images
 - 📊 Severity classification (Low / Medium / High)
 - 📄 AI-generated inspection reports
 - ⚠️ Risk assessment for industrial safety
 - 💬 Interactive GenAI chat assistant
 - 🔗 Multimodal AI fusion (Vision + LLM)
 - 📦 Containerized deployment (Docker-ready)
-

□ Tech Stack

Layer	Technology
Frontend	React, Vite
Backend	FastAPI
Computer Vision	PyTorch, YOLO / CNN / Vision Transformers
GenAI	Large Language Models (LLMs), Prompt Engineering
Deployment	Docker, Docker Compose
Language	Python

🚀 Deployment Architecture

- **Frontend:** React (Vite-based SPA)
- **Backend:** FastAPI serving:
 - Vision inference API
 - LLM reasoning API
- **Containerization:** Docker + Docker Compose
- **Communication:** REST APIs between frontend and backend

Corrosion Detection GenAI

An end-to-end multimodal system that combines a computer-vision model (CNN / YOLO / ViT) for detecting corrosion in images of metal structures with a Generative AI / LLM reasoning layer that produces human-readable inspection reports, severity assessments, maintenance recommendations, and an interactive chat ****Assistant****.

UI Overview

The frontend is a single-page React app with three panels:

1. ****Sidebar (left)**** - "Select Plant ID" / "Select Asset ID" dropdowns (populated dynamically from ``/plants`` and ``/plants/{id}/assets``) and an "Upload the Image" card with a "Browse here" button.
2. ****Image panel (center)**** - shows the uploaded inspection image with detected corrosion regions overlaid as bounding boxes.
3. ****Assistant (right)**** - a chat panel that automatically summarizes the detection results after an image is analyzed, and lets the user ask follow-up questions grounded in that result.

Project Structure

```

corrosion-detection-genai/
├── app/
│   ├── frontend/
│   │   ├── src/
│   │   │   ├── components/
│   │   │   │   ├── Header, PlantAssetSelector,
│   │   │   │   ├── ImageUploadCard,
│   │   │   │   ├── ImagePreview, AssistantPanel
│   │   │   │   ├── __tests__/
│   │   │   │   ├── Vitest + React Testing Library tests
│   │   │   │   ├── api.js
│   │   │   │   └── Backend API client (dynamic
│   │   └── VITE_API_URL)
│   └── App.jsx
│       ├── index.css
│       ├── .env.example
│       ├── package.json
│       ├── vite.config.js
│       └── Dockerfile
├── backend/
│   └── main.py
├── models/
│   ├── vision_model/
│   └── llm_module/
├── data/
│   ├── raw_images/
│   └── processed/
├── pipelines/
│   ├── preprocessing.py
│   ├── inference_pipeline.py
│   └── multimodal_fusion.py
├── utils/
│   ├── config_loader.py
│   ├── image_utils.py
│   ├── text_utils.py
│   └── logger.py
├── configs/
│   └── config.yaml
├── tests/
│   ├── conftest.py
│   └── test_api.py

```

React (Vite) UI

Header, PlantAssetSelector,

ImagePreview, AssistantPanel

Vitest + React Testing Library tests

Backend API client (dynamic

FastAPI service

CNN / YOLO / ViT for corrosion detection

GenAI / LLM prompts + chat reasoning

Central, dynamic config (+ env overrides)

All dynamic values: paths, model, LLM, # backend, plants/assets, chat prompts

pytest unit tests (backend + pipeline)

```

├── test_config_loader.py
├── test_image_utils.py
├── test_text_utils.py
├── test_vision_model.py
├── test_llm_reasoning.py
├── test_multimodal_fusion.py
├── test_inference_pipeline.py
├── notebooks/
│   └── exploration.ipynb
├── docs/
│   ├── architecture.png
│   └── api_docs.md
├── pytest.ini
├── requirements.txt
├── Dockerfile                                # Backend image
├── docker-compose.yml                       # Backend + frontend
└── main.py                                  # CLI entrypoint
...

```

Configuration (everything is dynamic)

All runtime values live in ``configs/config.yaml``:

- ``vision_model`` - model type/weights/thresholds/classes/image size
- ``llm_module`` - LLM provider, model name, temperature, system prompt
- ``backend`` - host, port, title, CORS origins
- ``plants`` - the plant/asset registry shown in the UI dropdowns
- ``chat`` - the assistant's welcome message and system prompt

Any value can be overridden via environment variables using the ``CDG_<SECTION>_<KEY>`` convention (see ``utils/config_loader.py``), e.g.:

```

```bash
export CDG_BACKEND_PORT=9000
export CDG_VISION_MODEL_DEVICE=cuda
export CDG_LLM_MODULE_MODEL_NAME=claude-opus-4-7
export ANTHROPIC_API_KEY=sk-ant-...
```

```

The React frontend reads its backend URL from ``VITE_API_URL`` (``app/frontend/.env.example``), and also calls ``GET /config`` at startup to pick up the welcome message, model name, classes, etc.

Local Development

1. Backend (FastAPI)

```
```bash
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt

uvicorn app.backend.main:app --reload --port 8000
```
```

The API docs are available at `http://localhost:8000/docs`.`

2. Frontend (React + Vite)

```
```bash
cd app/frontend
cp .env.example .env # adjust VITE_API_URL if needed
npm install
npm run dev
```
```

Open `http://localhost:5173`.`

3. CLI (no UI)

```
```bash
python main.py --image data/raw_images/sample.jpg
```
```

Testing

Backend / pipeline unit tests (pytest)

```
```bash
pip install -r requirements.txt
pytest
```
```

Runs fully offline: the vision model uses a stub (no detections) if no weights file is present, and the LLM module uses a deterministic offline fallback if no API key is configured.

Frontend unit tests (Vitest + React Testing Library)

```
```bash
cd app/frontend
npm install
npm test
```
```

Deployment

Docker Compose (backend + frontend)

```
```bash
```

```
docker compose up --build
```

```
```
```

- Backend: ``http://localhost:8000``
- Frontend: ``http://localhost:5173``

Individual images

```
```bash
```

```
Backend
```

```
docker build -t corrosion-backend .
```

```
docker run -p 8000:8000 --env-file .env corrosion-backend
```

```
Frontend
```

```
docker build -t corrosion-frontend ./app/frontend \
```

```
--build-arg VITE_API_URL=https://your-backend-host
```

```
docker run -p 5173:5173 corrosion-frontend
```

```
```
```

API Reference

See ``docs/api_docs.md`` for full endpoint documentation:

- ``GET /health`` - health check
- ``GET /config`` - active configuration (models, classes, chat welcome message)
- ``GET /plants`` - list of plants
- ``GET /plants/{plant_id}/assets`` - assets for a plant
- ``POST /predict`` - upload an image (+ optional ``plant_id``/``asset_id``) → detections, risk level, AI inspection report
- ``POST /chat`` - ask the assistant a follow-up question, grounded in the latest ``/predict`` result for that plant/asset

How It Works

1. **Preprocessing** (``pipelines/preprocessing.py``) - resizes/normalizes images.
2. **Vision Model** (``models/vision_model/``) - detects corrosion regions, classifies severity, outputs bounding boxes + confidence scores.
3. **LLM Reasoning** (``models/llm_module/``) - generates a structured inspection report and powers the conversational Assistant.

```
4. **Multimodal Fusion** (`pipelines/multimodal_fusion.py`) - combines  
vision + LLM outputs into a single response.  
5. **FastAPI Backend** - exposes `/predict`, `/chat`, `/plants`, `/config`.  
6. **React Frontend** - three-panel UI (plant/asset selection + upload,  
image preview with overlay, assistant chat).  
  
## License  
MIT
```

□ Key Modules

1. Vision Module

- YOLO / CNN / ViT-based detection
- Outputs bounding boxes + confidence scores

2. LLM Reasoning Module

- Converts vision output → structured insights
- Generates:
 - Inspection summary
 - Risk level
 - Maintenance recommendations

3. Multimodal Fusion

- Combines image + detection + metadata
- Produces final AI inspection report

4. Chat Assistant

- Allows users to ask:
 - “How severe is the corrosion?”
 - “What maintenance is needed?”
 - “Is this asset safe for operation?”

▮ Output Example

Input: Industrial pipeline image

Output:

- Corrosion detected: Yes
- Severity: High
- Risk Level: Critical

- Recommendation:
 - Immediate inspection required
 - Apply anti-corrosion coating
 - Schedule maintenance within 7 days
-

Future Enhancements

- Real-time video stream inspection
 - Edge deployment (IoT cameras)
 - Fine-tuned domain-specific LLM
 - Integration with maintenance management systems (CMMS)
 - Mobile inspection app
-

Author

Pooja Sunkara